

CLOUD-NATIVE
SECURITY

eBPF: Revolutionizing Cloud-Native Security

From Kernel Visibility to Runtime Protection

By Alam Ahmed



March 2026
Technical Deep Dive





How would you describe your organization's current cloud-native security visibility?

Alam Ahmed

Junior Infrastructure Engineer

4 Years in Tech | Support → Networking → **Cloud Infrastructure** & Platform Engineering

focused on **Infrastructure & Cloud Security**

Speaker & Contributor

- **Elastic Observability London:**
Terraform and Elastic
- **Yorkshire DevOps:**
Security as code: OPA, Terraform, & AWS
- **contributed to envoy AI Gateway documentation**
- **cloud-custodian**
- **Yoke**

Certifications

Cilium (5)

Linux Fdn (7)

AWS (10)

Cisco (16)

Platform Eng (4)

Chainguard (2)

Core Stack

eBPF

Python

Go

AWS

Azure

Cilium

Tetragon

Kernel Security

Kubernetes

Terraform

AGENDA

Today's Journey

01

The Cloud-Native Security Crisis

Why traditional security approaches are failing in containerized environments and the deployment paradox organizations face

02

eBPF: The Game-Changing Primitive

Understanding the kernel-level technology that's reshaping observability with 10x performance improvements

03

The CNCF eBPF Ecosystem

Production-ready tools: Falco, Tetragon, Cilium, and the 41+ project ecosystem powering modern security

04

Building Security Observability Patterns

Practical implementations: process monitoring, network correlation, file integrity, and syscall profiling

05

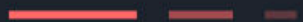
Integration & Real-World Impact

Connecting security data with observability platforms and proven hyperscale adoption at Meta, Google, Cloudflare

CHAPTER
01

The Cloud-Native Security Crisis

Why traditional security approaches are failing in
containerized environments



THE CRISIS

The Security-Deployment Paradox



67%

of
organizations

delay or slow down application deployment due to Kubernetes and container security concerns, according to Red Hat's 2024 State of Kubernetes Security report.

Source: Red Hat State of Kubernetes Security 2024



95%

by 2025

of new digital workloads will be deployed on cloud-native platforms, up from 30% in 2021. Gartner estimates this massive shift is already underway.

Source: Gartner Forecast 2024



The Disconnect

Organizations are caught in a paradox: they need the speed and flexibility of cloud-native architectures, but traditional security tools cannot keep pace with the dynamic, ephemeral nature of containers and microservices.



Security blocks innovation

67% slow down deployments due to security gaps



Blind spots create risk

60% worry about vulnerabilities and misconfigurations



Compliance challenges

42% cite inadequate security investment as top concern



The question: How do we achieve security without sacrificing the speed and agility that cloud-native promises?

THE PROBLEM

Traditional Security: The "Outside-Looking-In" Approach

Traditional perimeter-focused and host-centric security monitoring paradigms are architecturally misaligned with the dynamic, distributed, and ephemeral nature of cloud-native systems, preventing them from achieving the full depth of security, resilience, manageability, and observability defined by cloud-native principles.



Data Collection

- Agents installed in
 - containers
- Log scraping and parsing
- Network taps and packet inspection
- External vulnerability scanners



Detection Layer

- SIEM (Security Information & Event Management)
- Alert systems based on
 - logs/rules
- Signature-based detection
- Periodic scanning cycles



Key Limitations

- Reactive detection** – only after suspicious activity is logged
- No behavioral context** – misses anomalous patterns
- No kernel visibility** – blind to root-level threats
- Detection after disclosure** – attack already underway

ARCHITECTURAL MISMATCH

Why Traditional Approaches Fall Short



Dynamic vs. Static

Cloud-native: Containers spin up/down in seconds, IP addresses change constantly, microservices communicate ephemerally

Traditional tools: Assume static IPs, persistent hosts, predictable network boundaries. Cannot track rapidly changing workloads.



Ephemeral vs. Persistent

Cloud-native: Pods live minutes to hours, containers are immutable and replaceable, no persistent state

Traditional tools: Assume long-running servers with persistent storage. Forensics becomes impossible when containers vanish.



Distributed vs. Centralized

Cloud-native: Thousands of microservices across multiple nodes, service meshes, complex call chains

Traditional tools: Designed for monolithic applications with clear perimeters. Cannot trace requests across distributed systems.



The Visibility Gap

Traditional monitoring operates at the **application or network layer**, missing critical context from the kernel where attacks actually occur.

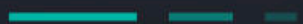
Without kernel visibility, you cannot see: privilege escalation attempts, container breakouts, rootkit installations, or syscall-level anomalies.

“ **The result:** Security teams are flying blind in cloud-native environments, relying on outdated tools that create more noise than signal. When attacks happen, detection comes too late—after the damage is done.

CHAPTER 02

eBPF: The Game-Changing Primitive

Understanding the technology that's reshaping kernel-level observability



What is eBPF?

eBPF (extended Berkeley Packet Filter) is a technology built into the Linux kernel that allows sandboxed programs to run safely inside the kernel itself. Originally designed for packet filtering, it has evolved into one of the most powerful infrastructure technologies for networking, observability, and security.



Safe & Sandboxed

Programs run in a restricted virtual machine within the kernel. The eBPF verifier ensures safety, preventing operations that could crash or compromise the system.



Kernel-Level Execution

Runs directly in the Linux kernel with minimal overhead. JIT compilation converts bytecode to native machine code for maximum performance.



Zero Code Changes

No application modification required. eBPF attaches to kernel events transparently, making it ideal for production environments.



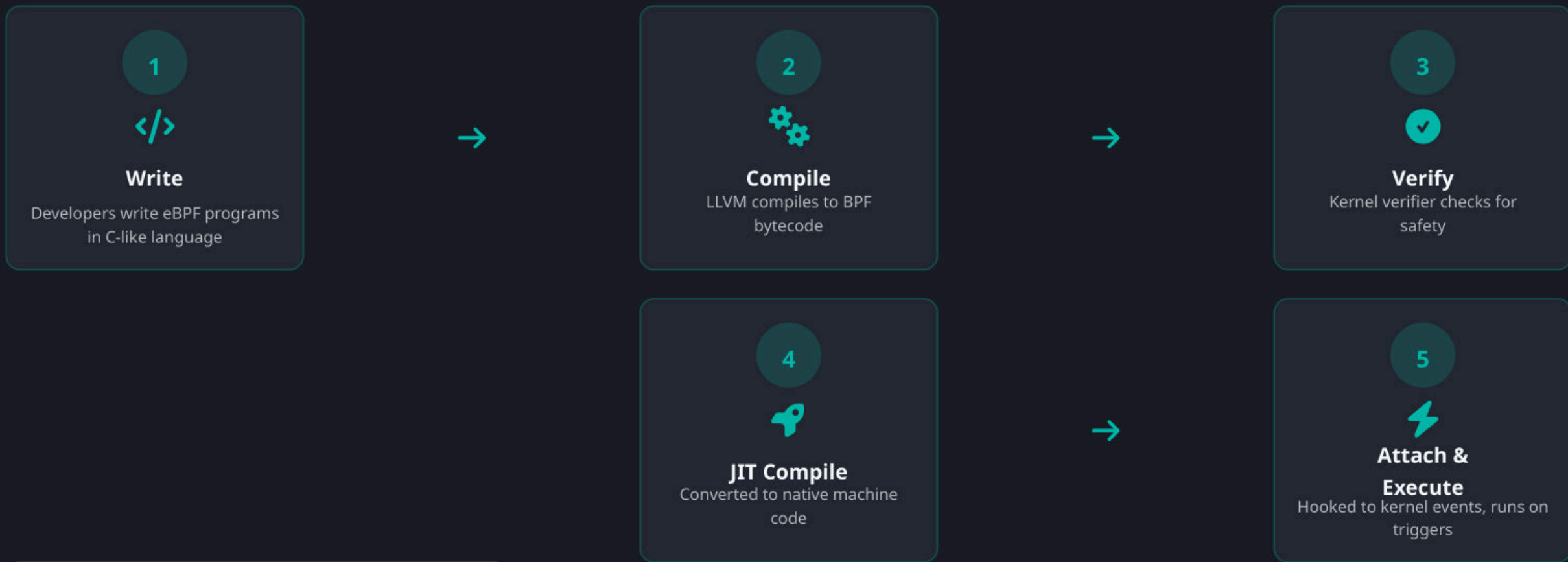
Container-Aware

Understands cgroups, namespaces, and Kubernetes metadata by nature. Provides container and pod context for every event.



Think of eBPF as: "JavaScript for the kernel" – it allows safe, dynamic programmability of kernel behavior without changing kernel source code or loading risky kernel modules.

How eBPF Works: The Execution Flow



The Verifier

Ensures programs cannot crash the kernel, run indefinitely, or access unauthorized memory. Provides safety guarantees before execution.



BPF Maps

Key-value data structures for storing state, sharing data between eBPF programs and user space, enabling complex logic and aggregation.



Attachment Points

eBPF can attach to: kprobes/kretprobes (kernel functions), tracepoints, XDP (network driver), tc (traffic control), socket filters, and more.

PERFORMANCE

eBPF Performance: The 10x Advantage



Facebook's research shows eBPF-based monitoring solutions achieve **up to 10x performance improvements** over traditional user-space methods.

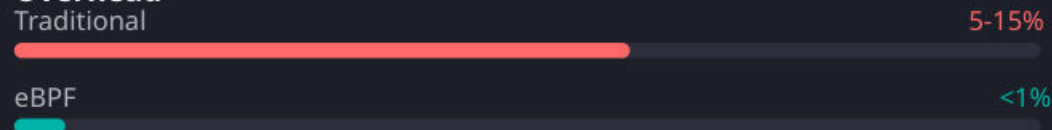
Why eBPF is So Efficient

- ✓ **No Context Switching**
Runs entirely in kernel space—no expensive user/kernel transitions
- ✓ **JIT Compilation**
Bytecode compiled to native machine code at load time
- ✓ **Event-Driven**
Only executes when triggered—no polling overhead

eBPF vs Traditional Methods

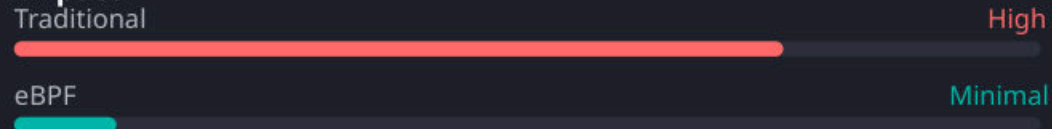
CPU

Overhead



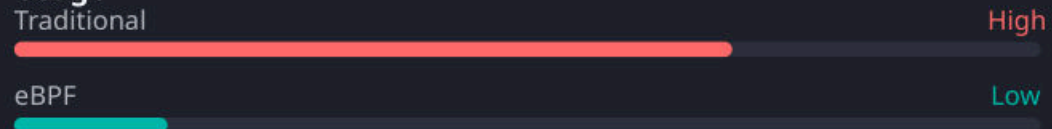
Latency

Impact



Memory

Usage



📖 **Cloudflare's XDP implementation:** Drops ~10 million packets per second on a single core, mitigating 3.8 Tbps DDoS attacks

Why eBPF is Perfect for Cloud-Native



Dynamic Attachment

eBPF programs can be loaded and unloaded **without system reboots**, enabling real-time updates to monitoring and security policies.

- ✓ Perfect for CI/CD pipelines and rapid deployment cycles



Kubernetes-Native

Understands **cgroups, namespaces, and Kubernetes metadata** by design. Automatically correlates kernel events with pods, containers, and services.

- ✓ No manual mapping of IPs to workloads



Zero Instrumentation

No code changes, no sidecars, no agents to inject. eBPF operates **transparently at the kernel level**, observing everything without application awareness.

- ✓ Works with any application, any language



Complete Lifecycle Visibility

From kernel syscalls to application behavior, eBPF provides **full-stack observability** with complete context—process, container, pod, and node.

- ✓ No blind spots, no gaps in visibility



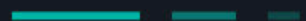
"eBPF catalyzes next-generation cloud native workloads because it expands a platform's capabilities, increases performance, and reduces complexity."

— Linux
Foundation
State of eBPF 2024

CHAPTER 03

The CNCF eBPF Ecosystem

Production-ready tools powering the next generation of security



The eBPF Ecosystem Landscape



Cilium

CNCF Graduated
Project

eBPF-based CNI providing networking, observability, and security.
Replaces iptables with high-performance kernel-level packet processing.

Networking

Observability

Security



Falco

CNCF Graduated
Project

Runtime security tool detecting unexpected behavior and threats in real-time. Monitors syscalls and Kubernetes audit logs.

Threat
Detection

Runtime
Security



Tetragon

Cilium Sub-project

Kernel-native security enforcement applying policies directly in eBPF.
Dramatically reduces events to user space.

Enforcement

Policy Engine



Pixie

CNCF Sandbox
Project

Zero-instrumentation observability for Kubernetes. Auto-instrumented telemetry without code changes.

Observability

Auto-
Instrumentation



41+

Projects on ebpf.io (up from 9 two years ago)

Ecosystem Growth
4.5x in 2 years

Falco: Runtime Threat Detection

Falco is a CNCF graduated project for detecting unexpected behavior and potential threats in real-time. Originally created by Sysdig, it's now used in production by organizations worldwide.

Real-Time Detection

Monitors system calls in real-time using eBPF or kernel modules

Custom Rules

Create tailored rules for your specific infrastructure needs

Extensive Rules

Out-of-the-box rules for common Kubernetes security scenarios

Lightweight

Designed for modern CI/CD pipelines and observability platforms

What Falco Detects



Container Escapes

Attempts to break out of container boundaries



Cryptojacking

Unauthorized crypto mining processes



Privilege Escalation

Attempts to gain elevated permissions



Data Exfiltration

Suspicious file and network activity

How Falco Works



Kernel Events

Syscalls via eBPF



Rule Engine

User-space filtering



Alerts

SIEM integration

Tetragon: Kernel-Native Enforcement

Tetragon is a Cilium sub-project that applies policy and filtering **directly in eBPF within the kernel**. This unique approach dramatically reduces the number of events sent to user space.

💡 **Key Differentiator:** While Falco filters in user space, Tetragon filters in the kernel—enabling massive performance gains.

Core Capabilities

- ✓ **Minimal Overhead**
eBPF enables deep observability with low performance impact
- ✓ **Kubernetes-Aware**
Recognizes workload identities: namespace, pod, labels
- ✓ **Kernel-Level Enforcement**
Blocks malicious activities at the kernel level, closing exploitation windows

Tetragon vs Falco: The Difference



Falco Approach

Collects all kernel events → sends to user space → filters and analyzes → generates alerts

Higher overhead | More events processed



Tetragon Approach

Applies policy in kernel → filters events at source → sends only relevant events to user space

Minimal overhead | Dramatically fewer events

📊 **Benchmark Result:** Tetragon uses significantly less CPU to monitor sensitive file access compared to other eBPF solutions

The background of the slide is a dark grey to black gradient, overlaid with a complex, light grey circuit board pattern. The pattern consists of numerous thin, interconnected lines and small circular nodes, resembling a microchip or a network diagram. The lines vary in thickness and form, creating a dense, technical aesthetic.

CHAPTER 04

Building Security Observability Patterns

Practical implementations for comprehensive runtime security

Process Execution Monitoring

eBPF captures every process execution at the kernel level with complete context—before user-space tampering can occur. This provides real-time visibility into what runs, where it runs, and how it was launched.



What Runs & Where

eBPF reads cgroup membership at execve time, resolving container, pod, and namespace context.

- ✓ Works across Docker, containerd, CRI-O
- ✓ Maps PID to Kubernetes metadata
- ✓ Real-time container context



How It Was Launched

Hooks execve/execveat syscalls to capture binary path and command-line arguments at execution time.

- ✓ Full command-line capture
- ✓ Binary path resolution
- ✓ Environment variables (optional)



Who Launched It

Tracks parent process chain to reconstruct full execution lineage and understand process relationships.

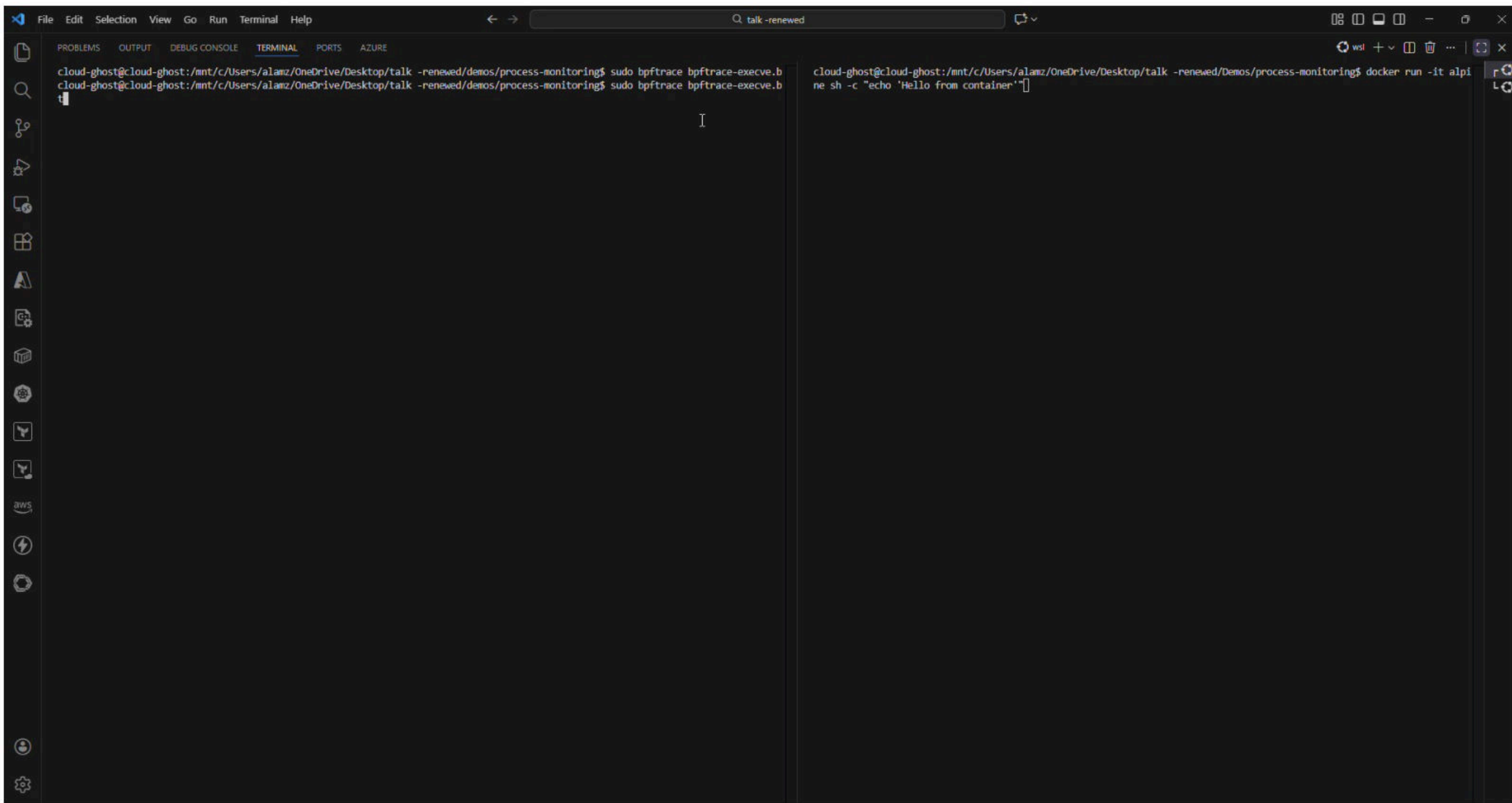
- ✓ Complete process tree
- ✓ User and group context
- ✓ Session and terminal info



This is eBPF in action. Every process execution is captured at the kernel level. Notice: real-time events, no application modification, full context (PID, command, executable path), and less than 1% CPU impact.

<1%

CPU Overhead



Network Security Correlation

eBPF answers three critical questions that traditional network monitoring cannot: Which container initiated this connection? What process is responsible? Can we correlate events in real-time?

1 Which Container?

eBPF maps sockets to cgroup IDs, then to specific containers and pods—providing immediate workload context.

→ Socket → cgroup ID → container → pod → namespace

2 What Process?

Kernel-level tracing connects TCP sockets directly to PID and process names, revealing which application initiated the connection.

→ Network event → PID → process name → executable path

3 Real-Time Correlation?

Yes—eBPF correlates network, process, and system call events atomically in the kernel, creating a unified security timeline.

→ Atomic event correlation with complete context

Example: Suspicious Outbound Connection

Traditional Monitoring

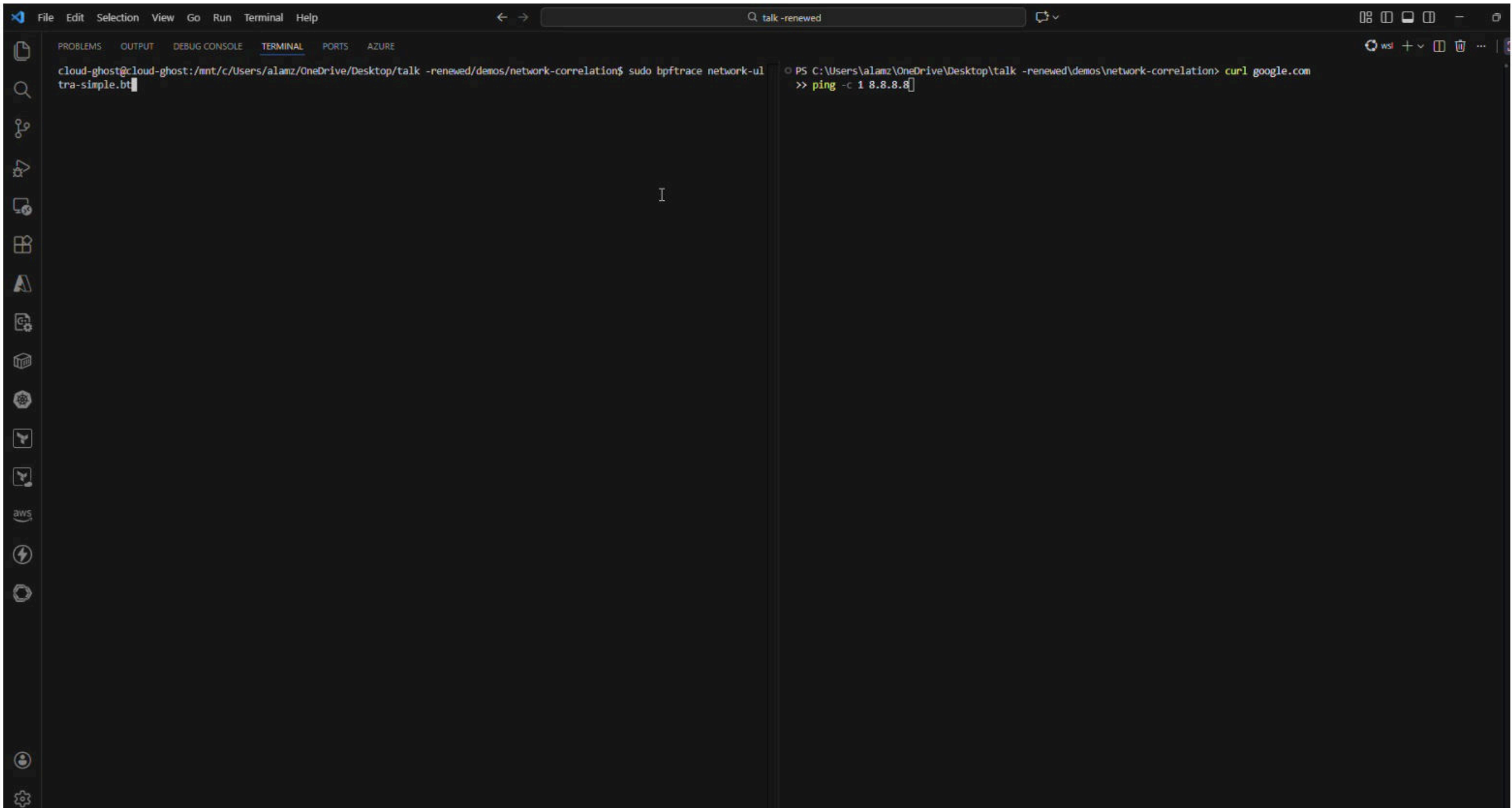
Connection from IP 10.0.1.15:443 → 185.220.101.42:8080

- ✗ No container context
- ✗ No process information
- ✗ Manual investigation required

eBPF-Enhanced Monitoring

nginx-pod-7d9f4b8c5-x2v9k (web-namespace) → 185.220.101.42:8080

- ✓ Container: nginx-pod-7d9f4b8c5-x2v9k
- ✓ Process: /usr/local/bin/nginx-worker
- ✓ Immediate threat assessment possible



File Integrity Monitoring with eBPF

eBPF-based file access monitoring reveals critical security context that traditional filesystem logs cannot provide: who accessed files, which container performed the access, and whether it was authorized.



Who Accessed?

eBPF reveals the exact user, process, and command that accessed sensitive files—providing complete attribution.

Traditional gap: Filesystem logs show WHAT but not WHO (process context)



Which Container?

eBPF maps cgroup ID to container name, pod, and namespace—essential context in multi-tenant environments.

Traditional gap: /var/log/secure doesn't know about containers



Was It Authorized?

eBPF validates file access against runtime security policies, detecting anomalous behavior in real-time.

Traditional gap: Static IAM policies vs. actual runtime behavior

Critical Security Questions Answered



Sensitive File Access
/etc/passwd, /etc/shadow, SSH keys, certificates



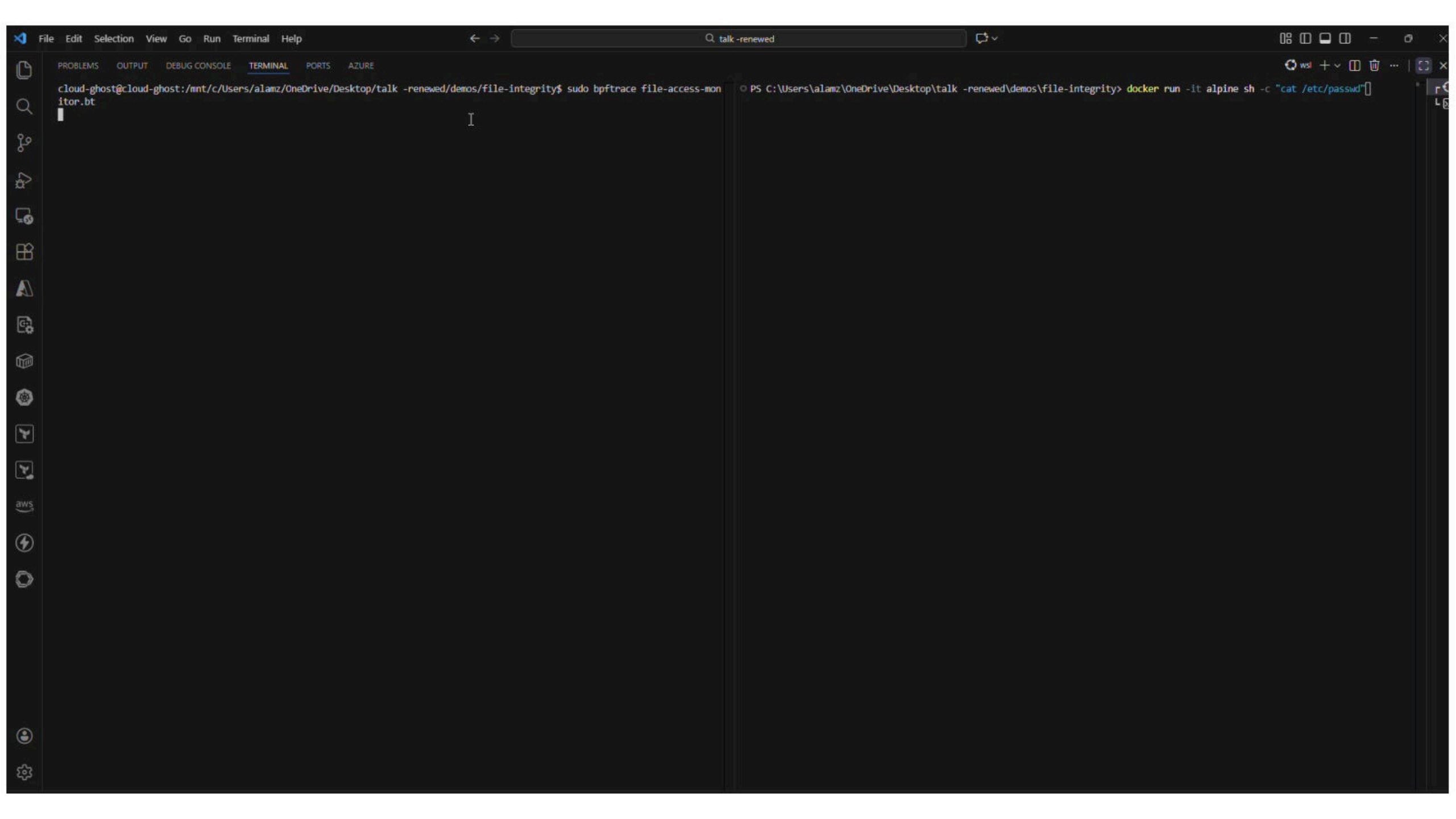
Configuration Changes
System configs, application settings, policy files



Binary Modifications
Executable changes, library injections, backdoors



Data Exfiltration
Database files, logs, sensitive data access patterns



System Call Profiling & Anomaly Detection

eBPF profiles syscalls to detect abnormal patterns, baseline container behavior, and identify potential container breakout attempts through suspicious syscall combinations.



Which Syscalls?

eBPF reveals real-time syscall frequency, arguments, and call chains with container context—impossible with traditional tools.

Traditional gap: strace can't map syscalls to containers



Abnormal Patterns?

eBPF detects deviations from baseline behavior, such as web servers suddenly making clone() or ptrace() calls.

Example: Web server spawning shells = compromise indicator



Container Breakout?

eBPF identifies suspicious syscall combinations that indicate container escape or privilege escalation attempts.

Signals: unshare(), mount(), chroot(), /proc access

Suspicious Syscall Patterns



Privilege Escalation

setuid(), setgid(), capset()



Container Escape

unshare(), mount(), pivot_root()



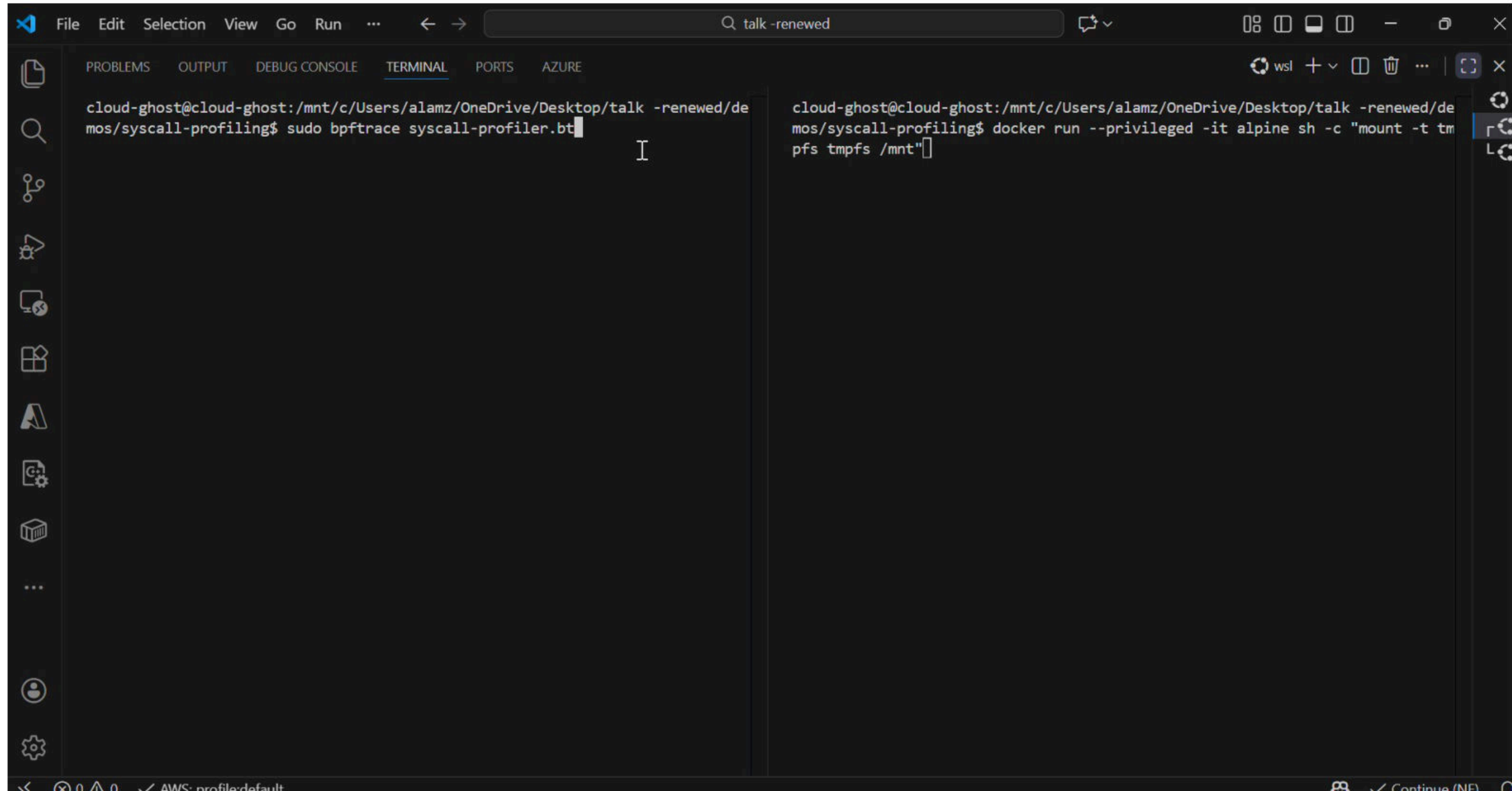
Process Injection

ptrace(), process_vm_writev()



Network Recon

socket(), connect() to unusual ports



CHAPTER 05

Integration & Real-World Impact

Connecting security data with cloud-native observability
platforms



The Power of Correlation: Complete Observability

eBPF enables connecting infrastructure metrics with security events in real-time, creating a unified observability platform that spans the entire cloud-native stack —from kernel to application.



Infrastructure Metrics

Collect performance data via eBPF for low-overhead observability



CPU & Memory

Per-process, per-container resource utilization



Network Throughput

Packet-level visibility with container context



Latency & I/O

System call latency, disk I/O patterns



Security Events

Enriched security telemetry from eBPF-powered tools



Suspicious Syscalls

Privilege escalation, container escape attempts



File Access Events

Sensitive file access with full context



Network Connections

Anomalous outbound connections



Prometheus/Grafana
Integration



Loki/Elasticsearch
Integration

Real-World Adoption & Impact

eBPF has been proven at hyperscale by the world's largest technology companies. For more than five years, eBPF has operated on millions of devices and servers worldwide—most people are already impacted by it without knowing.

**Meta**

Every Packet

Every single packet that goes in and out of Meta's datacenters is processed by eBPF. Katran, their XDP-based L4 load balancer, powers their entire infrastructure.

 Billions of packets per second**Google**

Kernel Security

Uses eBPF for **most of its kernel security** enforcement. eBPF powers their internal security monitoring and policy enforcement at scale.

 Production security enforcement**Cloudflare**eDoS
Mitigation

Auto-mitigated a **world record 3.8 Tbps DDoS attack** using XDP-based filters. Drops ~10 million packets per second on a single core.

 3.8 Tbps attack mitigated automatically**Android**

Every Device

Every Android phone uses eBPF to monitor traffic for data usage statistics, firewall rules, and power-saving network restrictions.

 3+ billion devices worldwide

Traditional vs eBPF-Native Security

The shift from **outside-looking-in** to **inside-looking-out** represents a fundamental change in how we approach cloud-native security—moving from reactive log analysis to proactive kernel-level visibility.



Traditional Security



Reactive Detection

Only after suspicious activity is logged



Log-Based Analysis

Dependent on application logging



High Overhead

5-15% CPU impact from agents



Limited Visibility

No kernel or container context



Context Switching

Expensive kernel/user transitions



eBPF-Native Security



Real-Time Detection

Kernel-level event capture



Kernel-Level Visibility

Complete system visibility
Minimal



Overhead

<1% CPU impact



Complete Context

Process, container, pod, node



Kernel-Space Execution

No context switching overhead



10x Performance Improvement

Facebook's benchmark: eBPF vs traditional monitoring

10x

faster & more
efficient

KEY TAKEAWAYS

The Future of Cloud-Native Security is Here

1

The Security Crisis is Real

Traditional security approaches are architecturally misaligned with cloud-native systems. 67% of organizations delay deployments due to security concerns.

2

eBPF is the Game-Changer

Kernel-level visibility with minimal overhead and zero code changes. 10x performance improvements over traditional methods with <1% CPU impact.

3

Ecosystem is Production-Ready

CNCF projects like Falco, Tetragon, and Cilium provide enterprise-grade security. The ecosystem grew from 9 to 41+ projects in just two years.

4

Comprehensive Runtime Security

Process monitoring, network correlation, file integrity, and syscall profiling—all with complete container and Kubernetes context.



Proven at Hyperscale

Meta, Google, Cloudflare, and billions of Android devices prove eBPF is ready for enterprise deployment. The technology that powers the hyperscalers is available to everyone.

95%

workloads cloud-native by
2025

The question is no longer "Should we use eBPF?" but **"How quickly can we adopt it?"**

Scan to connect with me on LinkedIn

